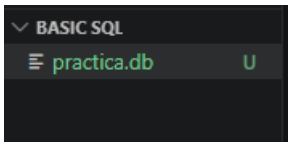
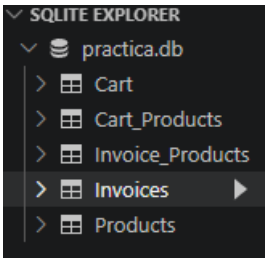
1. Cree una nueva base de datos en SQLite.

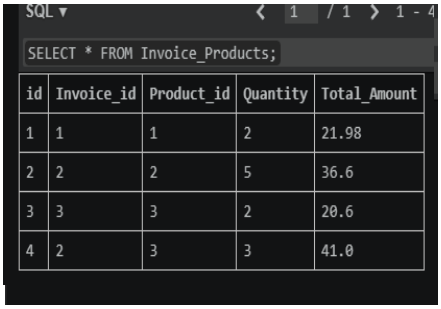


2.Replique las tablas creadas anteriormente en 🔗 **Ejercicio de Bases de Datos**, con sus respectivos PKs, FKs, constraints, y demás requerimientos.



1. Investigue cómo hacer que los PKs se generen **automáticamente**.
   Para que esto suceda se digita el siguiente código INTEGER PRIMARY KEY AUTOINCREMENT, esto genera el ID automáticamente, nunca reutiliza IDs eliminados, siempre sube.

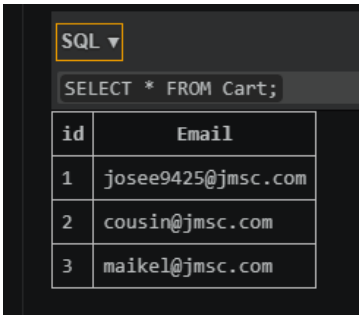
2. Utilice los tipos de datos adecuados.



SQL ▾ 1 / 1 1 - 4

```
SELECT * FROM Invoice_Products;
```

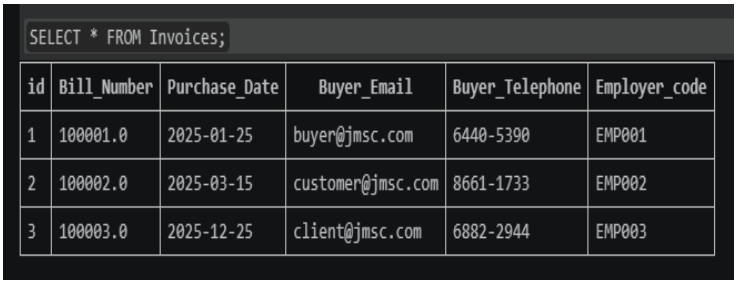
id	Invoice_id	Product_id	Quantity	Total_Amount
1	1	1	2	21.98
2	2	2	5	36.6
3	3	3	2	20.6
4	2	3	3	41.0



SQL ▾

```
SELECT * FROM Cart;
```

id	Email
1	josee9425@jmsc.com
2	cousin@jmsc.com
3	maikel@jmsc.com



```
SELECT * FROM Invoices;
```

id	Bill_Number	Purchase_Date	Buyer_Email	Buyer_Telephone	Employer_code
1	100001.0	2025-01-25	buyer@jmsc.com	6440-5390	EMP001
2	100002.0	2025-03-15	customer@jmsc.com	8661-1733	EMP002
3	100003.0	2025-12-25	client@jmsc.com	6882-2944	EMP003

```
SELECT * FROM Products;
```

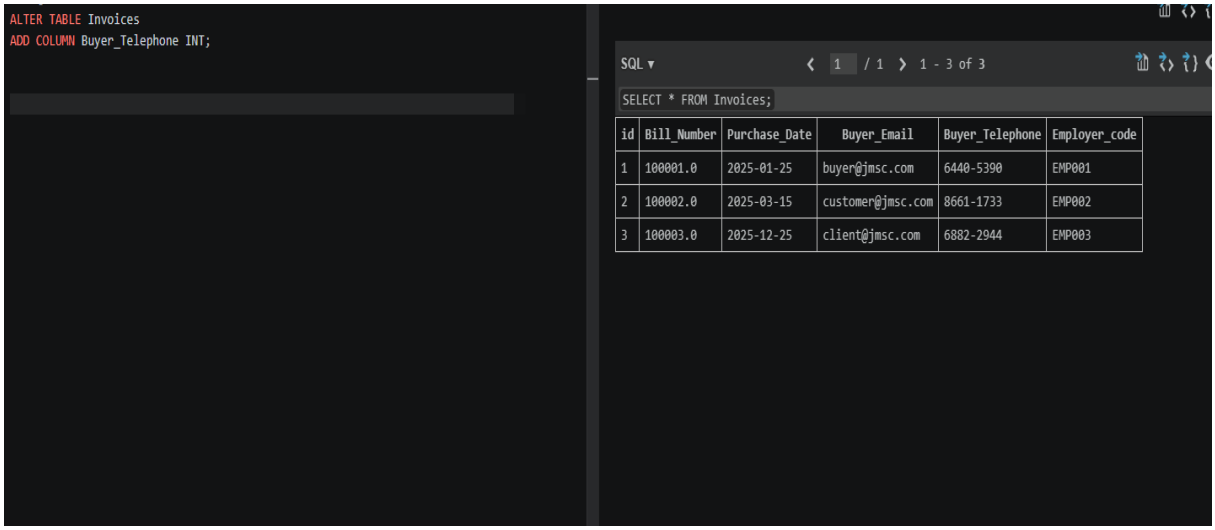
id	Product_Code	Name	Price	Entry_Date	Brand	Stock
1	B001	The Great Gatsby	10.99	2023-01-15	Scribner	100
2	B002	1984	12.99	2023-01-16	Secker & Warburg	150
3	B003	The Three-Body Problem	15.99	2023-01-17	Tor Books	200

3. Si existe alguna limitante por SQLite, documéntela y resuelva la limitante como considere.

Si al crear la tabla se utiliza una coma en la última columna, da un error “near)”.

El mismo error se presenta si no se cierran paréntesis como en el caso de VARCHAR(100.

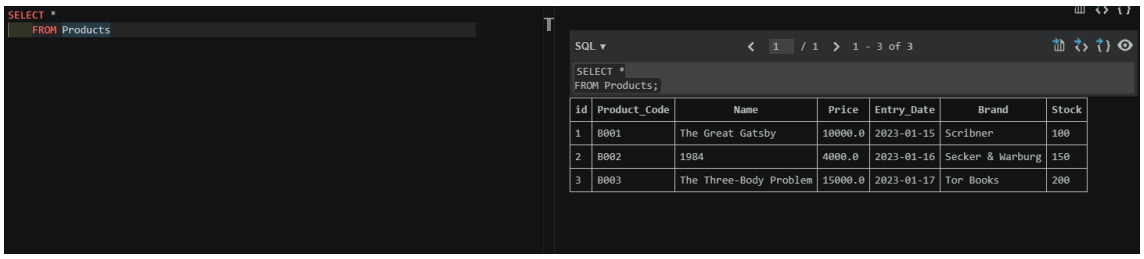
3 Utilizando el comando ALTER, modifique la tabla de Facturas y agregue una columna para almacenar también el número de teléfono del comprador, y otra para el código de empleado del cajero que realizó la venta.



id	Bill_Number	Purchase_Date	Buyer_Email	Buyer_Telephone	Employer_code
1	100001.0	2025-01-25	buyer@jmsc.com	6440-5390	EMP001
2	100002.0	2025-03-15	customer@jmsc.com	8661-1733	EMP002
3	100003.0	2025-12-25	client@jmsc.com	6882-2944	EMP003

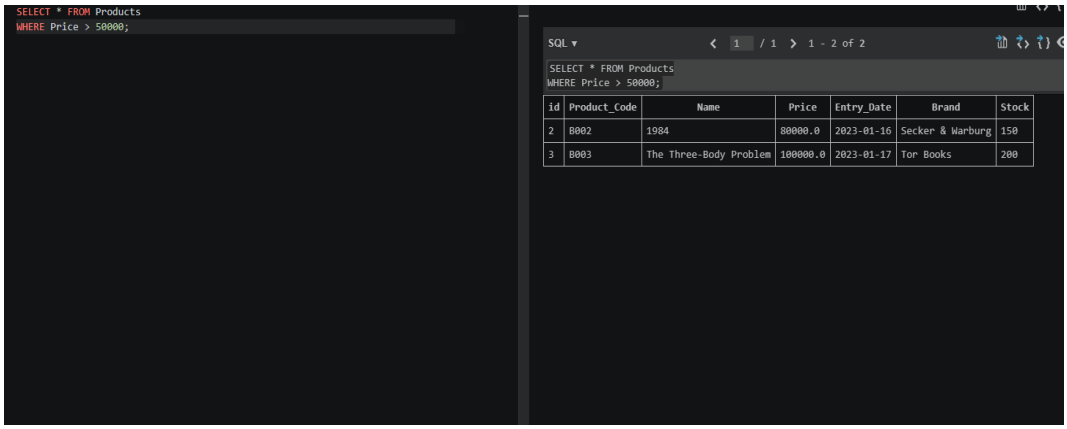
4.Realice los siguientes SELECT:

1. Obtenga todos los productos almacenados:



id	Product_Code	Name	Price	Entry_Date	Brand	Stock
1	B001	The Great Gatsby	10000.0	2023-01-15	Scribner	100
2	B002	1984	4000.0	2023-01-16	Secker & Warburg	150
3	B003	The Three-Body Problem	15000.0	2023-01-17	Tor Books	200

2. Obtenga todos los productos que tengan un precio mayor a 50000:



id	Product_Code	Name	Price	Entry_Date	Brand	Stock
2	B002	1984	80000.0	2023-01-16	Secker & Warburg	150
3	B003	The Three-Body Problem	100000.0	2023-01-17	Tor Books	200

3. Obtenga todas las compras de un mismo producto por id.

```
SELECT Product_id, COUNT(id) FROM Invoice_Products
GROUP BY Product_id;
```

SQL ▾ 1 / 1 1 - 3 of 3

```
SELECT Product_id, COUNT(id) FROM Invoice_Products
GROUP BY Product_id;
```

Product_id	COUNT(id)
1	1
2	1
3	2

4. Obtenga todas las compras agrupadas por producto, donde se muestre el total comprado entre todas las compras.
   UNION TOTAL--- se utiliza para unir ambos comandos

```
SELECT Invoice_id, SUM(Total_Amount) as Total_Amount
FROM Invoice_Products
GROUP BY Invoice_id

UNION ALL

SELECT "GRAN TOTAL", SUM(Total_Amount)
FROM Invoice_Products;
```

SQL ▾

Invoice_id	Total_Amount
1	60000.0
2	700000.0
3	200000.0
GRAN TOTAL	960000.0

5. Obtenga todas las facturas realizadas por el mismo comprador
   Se utiliza JOIN para unir la tabla de Cart con la de Invoice_Product y se le indica que Invoice_id va ser igual al id de Cart.

```
SELECT
    Cart.Email,
    Invoice_Products.Invoice_id,
    Invoice_Products.Total_Amount
FROM Invoice_Products
JOIN Cart ON Invoice_Products.Invoice_id = Cart.id
ORDER BY Cart.Email;
```

SQL ▾ 1 / 1

```
SELECT
Cart.Email,
Invoice_Products.Invoice_id,
Invoice_Products.Total_Amount
FROM Invoice_Products
JOIN Cart ON Invoice_Products.Invoice_id = Ca
ORDER BY Cart.Email;
```

Email	Invoice_id	Total_Amount
cousin@jmsc.com	2	400000.0
cousin@jmsc.com	2	300000.0
josee9425@jmsc.com	1	60000.0
maikel@jmsc.com	3	200000.0

6. Obtenga todas las facturas ordenadas por monto total de forma descendente

```
SELECT * FROM Invoice_Products
ORDER BY Total_Amount DESC;
```

SQL ▾ 1 / 1 1 - 4

```
SELECT * FROM Invoice_Products
ORDER BY Total_Amount DESC;
```

id	Invoice_id	Product_id	Quantity	Total_Amount
2	2	2	5	400000.0
4	2	3	3	300000.0
3	3	3	2	200000.0
1	1	1	2	60000.0

7.Obtenga una sola factura por número de factura.